

Metro Ticket Booking System

Problem & Solution Specification (Project Requirement & Design Phase)

TARGET PLATFORM

ServiceNow App Engine

DOCUMENT TYPE

Requirement & Design Specification

APPLICATION SCOPE

x_snc_metro_book

1. High-Level Problem & Solution Matrix

#	Domain / Area	Problem Statement	Technical Impact	ServiceNow Design Solution
P1	Turnstile Gate Latency	Hardware scanners require real-time validation (<200ms) to prevent physical queues at station gates.	Latency leads to commuter bottlenecks and gate hardware timeouts.	Implement an optimized Scripted REST API with lightweight JSON payloads and indexed queries on <code>sn_metro_ticket</code> .
P2	Fraud & Duplication	Commuters might screenshot QR codes and share them with non-paying passengers.	Loss of metro revenue and unauthorized entries.	Implement dynamic, Time-based Cryptographic Tokens (TOTP) in QR payload signed by Script Include.
P3	Offline Access	Commuters underground lose cellular/Wi-Fi coverage, preventing online ticket rendering.	Commuters cannot exit gates if the ticket fails to load.	Cache active ticket payloads locally within Now Mobile App Engine / Mobile SDK for offline canvas rendering.
P4	Payment / Ticket Mismatch	Gateway deducts money, but platform transaction fails midway due to network drops.	High volume of support tickets and commuter dissatisfaction.	Idempotent Webhook Processing via Flow Designer : Automated workflow triggers API refund via Integration Hub.
P5	Peak Hours Concurrency	Thousands of tickets booked simultaneously during rush hours overwhelm DB locks.	Database record locking and slow portal response times.	Write ticket records asynchronously using Async Business Rules and queued Flow Designer execution .
P6	Fare Complexity	Calculating fares across multi-line transfers, zones, or peak vs. off-peak hours dynamically.	Hardcoded scripts become unmaintainable when tariffs update.	Store fare matrices in <code>sn_metro_fare_matrix</code> and build a configurable Decision Table Engine .

2. Detailed Architectural Designs

Design 1: Low-Latency Gate Validation Engine (Solving Gate Queues)

The Problem: Metro turnstiles require response times under **200ms** to unlock physical gates. Standard database queries with joins or heavy business rules can take 500ms–1.5s, causing physical bottlenecks.

The ServiceNow Solution Design:

- **Inbound Scripted REST API:** Expose a dedicated endpoint `/api/x_metro/gate_validation/scan`.
- **Database Indexing:** Ensure `sn_metro_ticket` has composite indexes on `sys_id`, `state`, and `qr_hash`.

- **Lightweight Script Include:** Execute direct, lightweight GlideRecord calls with restricted field returns (setLimit(1)).
- **Asynchronous Log Offloading:** Respond to the gate with {"access": "ALLOW"} or {"access": "DENY"} immediately, offloading event auditing to an **Async Business Rule**.

```

[Gate Scanner]  ---> (HTTP POST) ---> [ Scripted REST API ]
                                   |
                                   Query Indexed Table (<50ms)
                                   |
                                   Validate Ticket State
                                   |
[Gate Opens]  <--- (JSON ALLOW) <--- Return Response (<150ms)
                                   |
                                   Async Event (Update Ticket to In-Transit)

```

Design 2: Fraud Prevention via Cryptographic QR Tokens (Solving Ticket Sharing)

The Problem: Static QR code screenshots enable multiple passengers to travel on a single ticket purchase.

The ServiceNow Solution Design:

- **Server-Side Token Generation:** Upon payment, a Script Include generates an encrypted HMAC token combining: Ticket_ID + Source_Station + Dest_Station + Timestamp + Salt.
- **Dynamic Refresh:** The mobile widget rotates an internal time-based hash every 30 seconds (TOTP).
- **Single-Use Lifecycle State Machine:** Draft → Active → In-Transit (scanned at entry) → Closed (scanned at exit) → Expired. Gate REST API rejects scans if ticket state is already In-Transit or Closed.

Design 3: Idempotent Payment & Auto-Refund Architecture (Solving Payment Failures)

The Problem: Network drops during gateway callbacks cause payment deduction without ticket generation.

The ServiceNow Solution Design:

- **Integration Hub Webhook Listener:** Listens for asynchronous payment gateway callbacks.
- **Idempotency Key Check:** Prevents duplicate handling using the gateway's unique transaction ID (txn_id) against sn_metro_payment.
- **Fallback & Auto-Refund Workflow:** A background Flow waits 60 seconds after payment initiation. If the ticket remains in Draft, Flow Designer triggers an outbound **Integration Hub REST Step** to refund the payment automatically.

Design 4: Dynamic Tariff & Route Calculation (Solving Pricing Complexity)

The Problem: Dynamic tariffs (peak hours, weekend discounts, distance zones) make hardcoded scripts difficult to maintain.

The ServiceNow Solution Design:

- **Data Schema:** sn_metro_station (stations/lines) & sn_metro_fare_matrix (base pricing).
- **ServiceNow Decision Tables:** Configure Decision Rules evaluating Base Fare, Time of Day, and User Category to output the final fare without code changes.

3. Design Phase Key Artifact Summary

Feature / Challenge Area	Primary ServiceNow Artifact	Secondary ServiceNow Artifact
Gate Validation	Scripted REST API (/api/x_metro/...)	Script Include (Class Library)
Fare Calculation	Decision Tables Engine	sn_metro_fare_matrix Table
Auto-Refund Flow	Flow Designer Workflow	Integration Hub REST Step
QR Code Generation	Custom Service Portal Widget	Canvas/JS QR Library
Ticket Expiration	Scheduled Script Execution (Job)	Flow Designer Timer Trigger
Commuter Experience	Service Portal & Now Mobile App	Service Catalog Items